

A Universal Two-Filter Machine (Yes / No events)

Any stream of events that is either **Yes** or **No** becomes a string of bits: **Yes = 1**, **No = 0**. This same machine fits anything — a Renko brick (up/down), a fixed slice (target/stop first), a coin, a daily habit, a test pass/fail.

Filter 1 (F1) — find the spot. When the bit string **ends** in the F1 pattern, that event is a **signal spot**. The **next** event is that signal's result (**Yes = hit = 1** / **No = miss = 0**). Those results, in order, build the **outcome string**.

Filter 2 (F2) — decide to act. When the **outcome string** ends in the F2 pattern, we are **READY**. A signal spot that is ready = **ACT**. The real result of acting shows up on the **next event** (the **action result** row).

Worked example — F1 = 10 F2 = 01 or 10

event #	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	full string
Yes / No	Y	N	Y	N	N	Y	Y	N	Y	N	N	Y	N	Y	Y	N	
bit (raw)	1	0	1	0	0	1	1	0	1	0	0	1	0	1	1	0	1010 0110 1001 0110
F1 spot		▼		▼				▼		▼			▼			▼	
signal result			Y(1)		N(0)				Y(1)		N(0)			Y(1)			1010 1
action								ACT		ACT			ACT			ACT	
action result									1		0			1			101

raw string = **1010011010010110** → (F1 finds each **10**, next event = result) → outcome string = **10101** → (F2 arms) → ACT at events 8, 10, 13, 16 → action-result string = **101**.

Reading it: the very first ▼ spots are only "wait" (not ready yet). Once the outcome string first ends in **01** or **10**, we are **READY**, and every later spot becomes an **ACT**. Each ACT's true result is the very next event in the **action result** row.

How to map it to your world. Renko: Yes = up brick, No = down brick. Fixed slice: Yes = target hit first, No = stop hit first. A habit: Yes = did it, No = skipped. The F1 spot and the F2 ready-signal are computed purely from the Yes/No pattern — the string runs on its own. Whether **acting** in the real world succeeds also depends on real-world conditions (in trading: live data feed, margin, broker fills, regulations, connection).